

# LAB ASSIGNMENT-6.5

**NAME:** YEROJU ANJALI

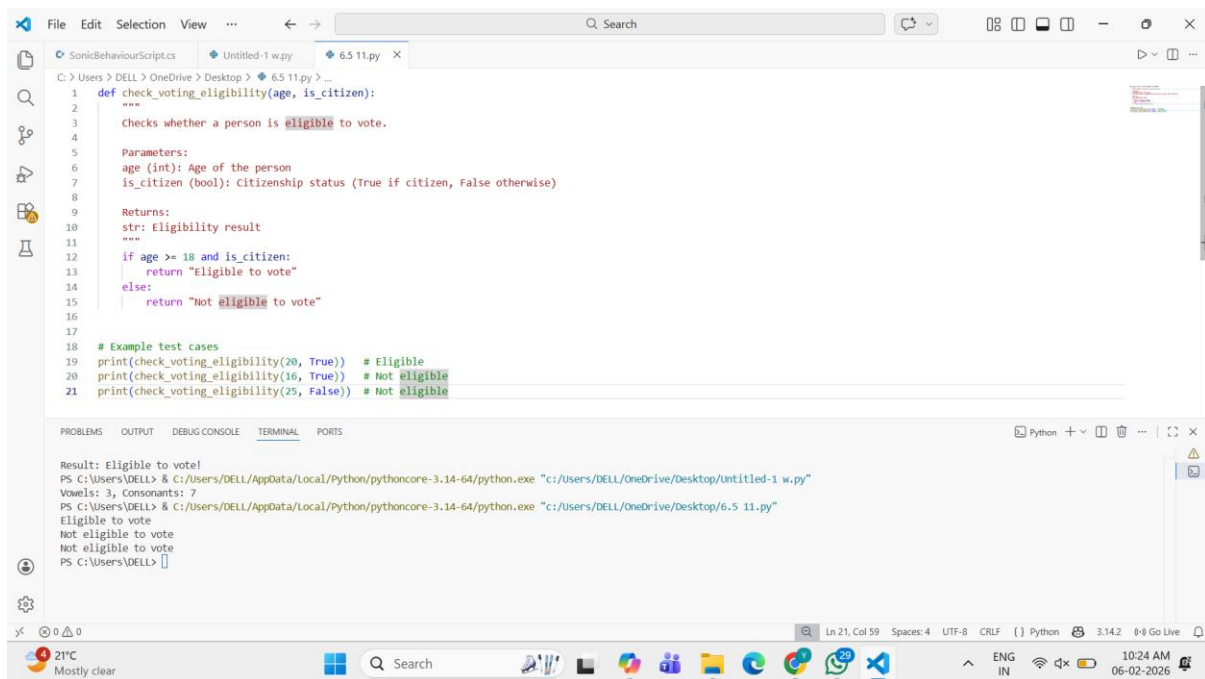
**HT.NO:** 2303A51416

**BT.NO:** 29

**TASK-01:** (AI-Based Code Completion for Conditional Eligibility Check)

**PROMPT:** “Generate Python code to check voting eligibility based on age and citizenship.”

## CODE AND OUTPUT :



```
def check_voting_eligibility(age, is_citizen):  
    """  
    Checks whether a person is eligible to vote.  
    Parameters:  
    age (int): Age of the person  
    is_citizen (bool): Citizenship status (True if citizen, False otherwise)  
    Returns:  
    str: Eligibility result  
    """  
    if age >= 18 and is_citizen:  
        return "Eligible to vote"  
    else:  
        return "Not eligible to vote"  
  
# Example test cases  
print(check_voting_eligibility(20, True)) # Eligible  
print(check_voting_eligibility(16, True)) # Not eligible  
print(check_voting_eligibility(25, False)) # Not eligible
```

```
Result: Eligible to vote!  
PS C:\Users\DELL> & C:/Users/DELL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/DELL/OneDrive/Desktop/Untitled-1 w.py"  
Vowels: 3, Consonants: 7  
PS C:\Users\DELL> & C:/Users/DELL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/DELL/OneDrive/Desktop/6.5 11.py"  
Eligible to vote  
Not eligible to vote  
Not eligible to vote  
PS C:\Users\DELL>
```

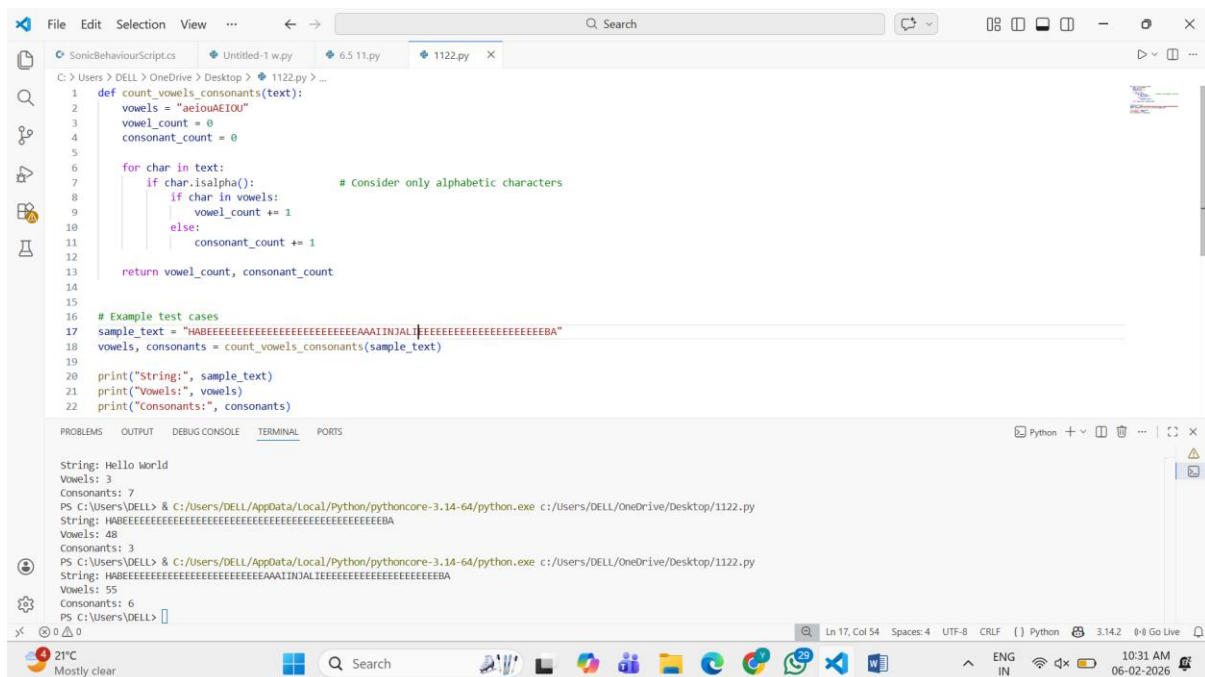
**OBSERVATION:** The AI-generated code effectively uses conditional statements to check voting eligibility by verifying both age and citizenship. By applying the logical AND operator, the program ensures that a person is eligible

to vote only when all required conditions are satisfied. The logic is clear, accurate, and easy to understand, leading to correct eligibility decisions for different input cases.

## TASK-02: (AI-Based Code Completion for Loop-Based String Processing)

**PROMPT:** “Generate Python code to count vowels and consonants in a string using a loop.”

## CODE AND OUTPUT:



```
1 def count_vowels_consonants(text):
2     vowels = "aeiouAEIOU"
3     vowel_count = 0
4     consonant_count = 0
5
6     for char in text:
7         if char.isalpha():             # Consider only alphabetic characters
8             if char in vowels:
9                 vowel_count += 1
10            else:
11                consonant_count += 1
12
13    return vowel_count, consonant_count
14
15
16 # Example test cases
17 sample_text = "HABEEEEEEEEEEEEEEEEEEEEAAAIINJALTEEEEEEEEEEEEEEEEEEBBA"
18 vowels, consonants = count_vowels_consonants(sample_text)
19
20 print("String:", sample_text)
21 print("Vowels:", vowels)
22 print("Consonants:", consonants)
```

String: Hello World  
Vowels: 3  
Consonants: 7  
PS C:\Users\DELL> & C:\Users\DELL\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/DELL/OneDrive/Desktop/1122.py  
String: HABEEEEEEEEEEEEEEEEEEEEAAAIINJALTEEEEEEEEEEEEEEEEEEBBA  
Vowels: 48  
Consonants: 3  
PS C:\Users\DELL> & C:\Users\DELL\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/DELL/OneDrive/Desktop/1122.py  
String: HABEEEEEEEEEEEEEEEEEEEEAAAIINJALTEEEEEEEEEEEEEEEEEEBBA  
Vowels: 55  
Consonants: 6  
PS C:\Users\DELL>

## OBSERVATION:

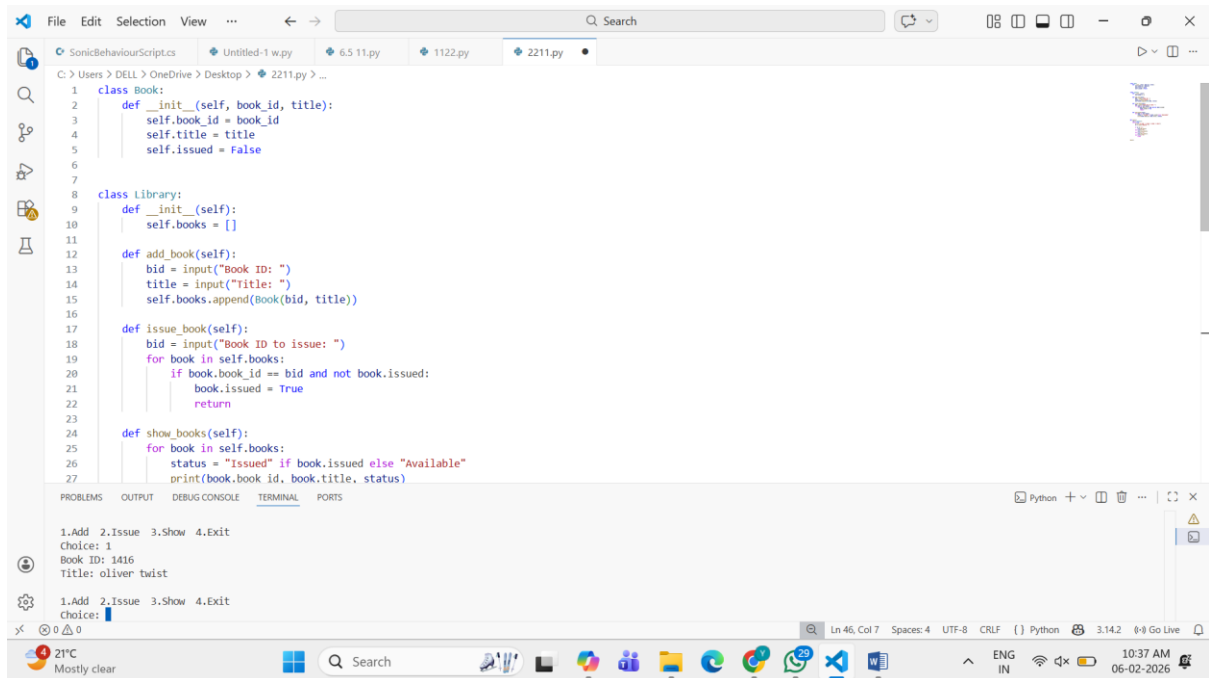
The AI-generated code efficiently processes a string using a loop to count vowels and consonants. It correctly iterates through each character, checks whether it is an alphabet, and classifies it as a vowel or consonant using conditional logic. The approach produces accurate counts while ignoring non-alphabetic characters, ensuring reliable output. Overall, the solution is clear,

logically structured, and demonstrates effective use of loops and conditions for string processing.

## TASK-03: (AI-Assisted Code Completion Reflection Task)

**PROMPT:** “Generate a Python program for a library management system using classes, loops, and conditional statements.”

## CODE AND OUTPUT:



The screenshot displays a Python IDE with a file named '2211.py'. The code defines two classes: 'Book' and 'Library'. The 'Book' class has attributes 'book\_id', 'title', and 'issued'. The 'Library' class has a list 'books' and methods 'add\_book', 'issue\_book', and 'show\_books'. The 'add\_book' method prompts for a book ID and title, and adds a new book to the library. The 'issue\_book' method prompts for a book ID to issue, checks if the book is available, and marks it as issued. The 'show\_books' method displays the status of all books in the library. The terminal output shows the user interacting with the program, choosing to add a book, entering '1416' for the ID and 'oliver twist' for the title, and then choosing to issue a book.

```
1 class Book:
2     def __init__(self, book_id, title):
3         self.book_id = book_id
4         self.title = title
5         self.issued = False
6
7
8 class Library:
9     def __init__(self):
10        self.books = []
11
12    def add_book(self):
13        bid = input("Book ID: ")
14        title = input("Title: ")
15        self.books.append(Book(bid, title))
16
17    def issue_book(self):
18        bid = input("Book ID to issue: ")
19        for book in self.books:
20            if book.book_id == bid and not book.issued:
21                book.issued = True
22                return
23
24    def show_books(self):
25        for book in self.books:
26            status = "Issued" if book.issued else "Available"
27            print(book.book_id, book.title, status)
```

1.Add 2.Issue 3.Show 4.Exit  
Choice: 1  
Book ID: 1416  
Title: oliver twist  
1.Add 2.Issue 3.Show 4.Exit  
Choice: 1

## OBSERVATION:

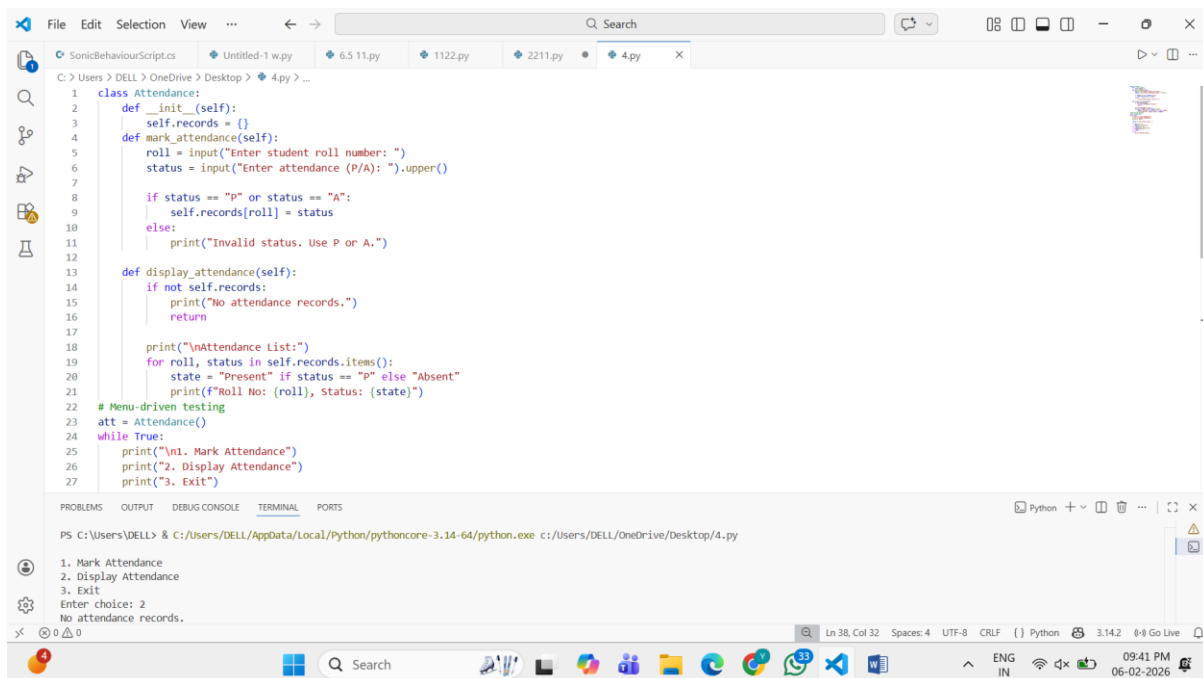
The AI-generated library management program effectively integrates classes, loops, and conditional statements to create a functional, menu-driven system. The class structure is clear and well-organized, making the code easy to understand and maintain. The use of loops enables continuous user interaction, while conditionals correctly handle different operations such as adding, issuing,

and displaying books. Overall, the AI suggestions are logically sound, practical, and provide a strong foundation that can be easily extended or improved with additional features.

## **TASK-04:** (AI-Assisted Code Completion for Class-Based Attendance System)

**PROMPT:** “Generate a Python class to mark and display student attendance using loops.”

### **CODE AND OUTPUT:**



```
1 class Attendance:
2     def __init__(self):
3         self.records = {}
4     def mark_attendance(self):
5         roll = input("Enter student roll number: ")
6         status = input("Enter attendance (P/A): ").upper()
7
8         if status == "P" or status == "A":
9             self.records[roll] = status
10        else:
11            print("Invalid status. Use P or A.")
12
13    def display_attendance(self):
14        if not self.records:
15            print("No attendance records.")
16            return
17
18        print("\nAttendance List:")
19        for roll, status in self.records.items():
20            state = "Present" if status == "P" else "Absent"
21            print(f"Roll No: {roll}, Status: {state}")
22
23    # Menu-driven testing
24    att = Attendance()
25    while True:
26        print("\n1. Mark Attendance")
27        print("2. Display Attendance")
28        print("3. Exit")
29        choice = input("Enter choice: ")
30
31        if choice == "1":
32            att.mark_attendance()
33        elif choice == "2":
34            att.display_attendance()
35        elif choice == "3":
36            break
```

PS C:\Users\DELL> & C:/Users/DELL/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/DELL/OneDrive/Desktop/4.py

1. Mark Attendance  
2. Display Attendance  
3. Exit  
Enter choice: 2  
No attendance records.

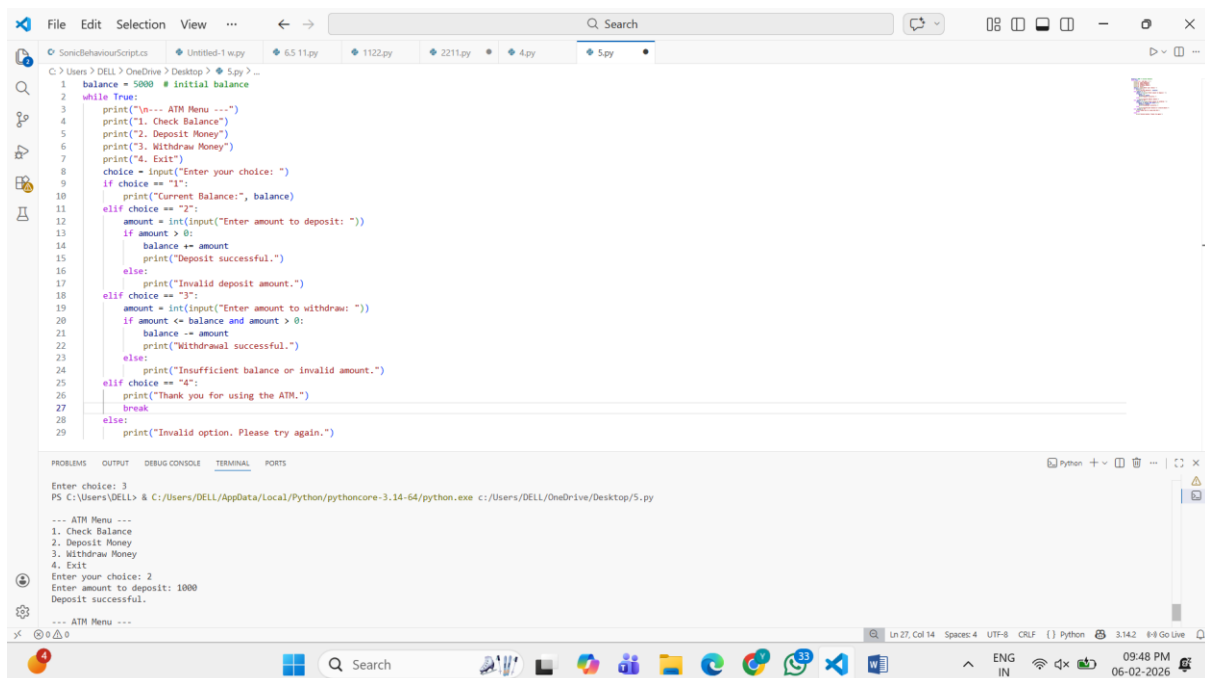
### **OBSERVATION:**

The AI-generated attendance system correctly uses a class to store and manage student attendance records. Loops are effectively applied to iterate through records for displaying attendance, while conditional statements ensure valid marking of present or absent status. The logic is simple, accurate, and produces correct attendance displays for different test cases. Overall, the AI provides a clear and reliable solution that demonstrates proper use of object-oriented concepts along with loops and conditionals.

## TASK-05: (AI-Based Code Completion for Conditional Menu Navigation)

**PROMPT:** “Generate a Python program using loops and conditionals to simulate an ATM menu.”

### CODE AND OUTPUT:



The screenshot displays a Visual Studio Code editor window with a Python file named '5.py'. The code implements an ATM menu simulation using a while loop and conditional statements. The initial balance is set to 5000. The menu options are: 1. Check Balance, 2. Deposit Money, 3. Withdraw Money, and 4. Exit. The program handles user input for choice and amount, performing calculations for deposits and withdrawals, and providing feedback messages. The terminal output shows the program running successfully, with the user entering choice 3 and depositing 1000, resulting in a successful deposit message.

```
1 balance = 5000 # initial balance
2 while True:
3     print("ATM Menu ---")
4     print("1. Check Balance")
5     print("2. Deposit Money")
6     print("3. Withdraw Money")
7     print("4. Exit")
8     choice = input("Enter your choice: ")
9     if choice == "1":
10        print("Current Balance:", balance)
11    elif choice == "2":
12        amount = int(input("Enter amount to deposit: "))
13        if amount > 0:
14            balance += amount
15            print("Deposit successful.")
16        else:
17            print("Invalid deposit amount.")
18    elif choice == "3":
19        amount = int(input("Enter amount to withdraw: "))
20        if amount <= balance and amount > 0:
21            balance -= amount
22            print("Withdrawal successful.")
23        else:
24            print("Insufficient balance or invalid amount.")
25    elif choice == "4":
26        print("Thank you for using the ATM.")
27        break
28    else:
29        print("Invalid option. Please try again.")
```

Enter choice: 3  
PS C:\Users\DELL> & C:\Users\DELL\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\Users\DELL\OneDrive\Desktop\5.py

--- ATM Menu ---  
1. Check Balance  
2. Deposit Money  
3. Withdraw Money  
4. Exit  
Enter your choice: 2  
Enter amount to deposit: 1000  
Deposit successful.

--- ATM Menu ---

### OBSERVATION:

The AI-generated ATM program effectively uses loops and conditional statements to simulate menu-based navigation. The loop ensures continuous user interaction, while conditional logic correctly handles options such as checking balance, depositing money, withdrawing money, and exiting the system. Input validation helps prevent invalid transactions, leading to accurate and reliable outputs. Overall, the AI solution is clear, well-structured, and successfully demonstrates conditional menu navigation.